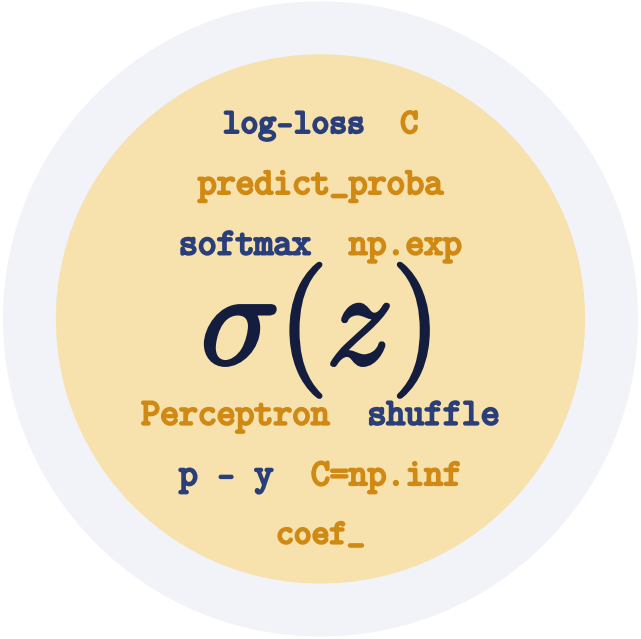
Логистическая регрессия и перцептрон

Шпаргалка



Формулы и вызовы из урока «Логистическая регрессия и перцептрон». Строка с формулой — что это, формула, ниже пример. Строка с кодом — что ищут, что делает, ниже пример и после стрелки его значение, снятое запуском. Примеры кода — на этих массивах:

```
import numpy as np
X = np.array([[1, 2], [2, 0], [4, 4], [5, 2]])
y = np.array([0, 0, 1, 1])
q = np.array([2, 2])
z = np.array([-2.0, 2.0])
```

Логистическая регрессия

Сигмоида — $\sigma(z) = \frac{1}{1+e^{-z}}$, $\sigma(-z) = 1 - \sigma(z)$, $\sigma' = \sigma(1-\sigma)$; $p = \sigma(w \cdot x)$, класс 1 при $w \cdot x > 0$
▷ $\sigma(\ln 3) = \frac{3}{4}$

Log-loss — $-\frac{1}{n}\sum_i \big(y_i \ln p_i + (1-y_i)\ln(1-p_i)\big)$, $p_i = \sigma(w \cdot x_i)$
▷ объект класса 1 с $p = \frac{3}{4}$: $\ln\frac{4}{3} \approx 0{,}288$

Его градиент — $\nabla L = \frac{1}{n} X^\top (p - y)$, у одного объекта $d\ell/dz = p - y$
▷ $x = (1, 2)$, класс 1, $p = \frac{3}{4}$: $\nabla\ell = -\frac{1}{4} \cdot (1, 2)$

Разделимые классы — минимума log-loss нет: веса растут без конца; спасает штраф
▷ `np.exp` переполняется уже при отступе около -710

`np.exp` — сигмоида: $1/(1+e^{-z})$

```
(1 / (1 + np.exp(-z))).round(3) → [0.119 0.881]
```

Перцептрон

Правило — классы ± 1; при $y_i(w \cdot x_i + b) \le 0$ поправка $w \leftarrow w + y_i x_i$, $b \leftarrow b + y_i$; проходы по порядку — до прохода без поправок

▷ $w = (1, 1)$, $b = 0$, объект $(1, 2)$ класса -1: $w = (0, -1)$, $b = -1$

Почему работает — поправка увеличивает $y_i(w \cdot x_i + b)$ на $\|x_i\|^2 + 1$; при разделимых классах поправок конечное число

▷ при неразделимых проходы ограничивают

Вызовы sklearn

Порядок — как у всех моделей: создать, `fit(X, y)`, `predict(X)`. Модуль — `sklearn.linear_model`.

`LogisticRegression` — штраф по умолчанию, $C = 1$; без штрафа `C=np.inf`

```
lg = LogisticRegression().fit(X, y)
```

`lg.predict_proba` — вероятности, по столбцу на класс

```
lg.predict_proba([q]).round(2) → [[0.7 0.3]]
```

`lg.coef_` — веса; свободный член — `intercept_`

```
lg.coef_.round(2) → [[0.86 0.52]]
```

`Perceptron` — перцептрон: `shuffle`, `tol`, `max_iter`

```
Perceptron().fit(X, y).predict([q]) → [0]
```

Где sklearn расходится с точными правилами

Логистическая — минимизирует $\frac{1}{2}\|w\|^2 + C\sum_i \ell_i$, своим способом спуска

▷ на разделимых данных вес конечен лишь из-за штрафа

Перцептрон — `shuffle=True` и останов по `tol`, а не по проходу без поправок

▷ «по порядку» — только своим кодом